Introduction to Node And Node.JS

- JS - was meant to run only in browser.
- following are some JS engines that are Present in modern day browser to run JavaScript.
- Node was developed by Ryan Dahl to run JS outside of the browser.
- Node = V8 Engine / Spider Monkey + Inbuilt Node Modules
- Node is a Runtime environment, that allows JS to run outside of browsers in local machines as well.

Now lets see some example to understand this concept of runtime environment :

- PlayStation games can run in a play station console only.
- I can make tandoori roti in a tandoor only.

Scanned with ACE Scanner

CJS (Common JavaScript)

```
// for importing
Const x = require ("module")
// for exporting
module.exports = x
```

- Common JS method is used to import or export any Node module.
- We are not using ES6 method of importing and exporting because almost all the industries in the market use the CJS method only, and we have to align with the same as most of the resources present on web also prefers the same.

Let us create two files in vs code
index.js & data.js

- Index.js File

Code

```
Const { sum, sub, pro, dir} = require("./data.js")
Sum (10, 10)
Sub (20, 5)
Pro ( 12, 12)
dir ( 10, 5)
```

Scanned with ACE Scanner

- data.js File

Code

```
Const Sum = (a,b) => {
      consale.log (a+b);
}
const Sub = (a,b) => {
      consale.log (a-b);
const Pro = (a,b) => {
      Consale.log (a*b);
}
Const div = (a,b) => {
      consale.log (a/b);
}
module.exports = {
    Sum,
    Sub,
    Pro,
    div
}
```

Now, when we run index.js by the help of node in our local machine then following is going to be the output

```
// 20
// 15
// 144
// 2
```

Scanned with ACE Scanner

Intraction with System

There are two methods by which we can interact with the System/machine :

- GUI ⇒ Graphical User Interface

  → This is the pretty common way that we use to interact with the system, that includes taking the help of graphics to demonstrate and visualise all the actions that we are doing.

- CLI ⇒ Command Line Interface

  → This kind of interaction mainly involves writing specific commands to perform any kind of operation in the system.

Common CLI commands

Please explore the following CLI commands, they are really fun :

1. ls → Directory listing
2. cd.. ⇒ Change to home directory
3. cd dir → Change directory to dir.

Scanned with ACE Scanner

Note :- (1.) JavaScript is often refered to as dynamically typed, concurrent, single-threaded and asynchronous.

- Dynamically Typed : You don't have to specify the data type of a variable when you declare it.
- Concurrent : In the context of website development, JavaScript can handle multiple events concurrently without blocking the execution of other code.
- Single-threaded : One task at a time.
- Asynchronous : It allows tasks to be executed independently of the main program flow, enabling non-blocking behaviour.

(2.) Module :- Modules are function only which we can export and use in other files.

Start with the project

- Create a index.js file.
- Create a text.txt file.
- Write some content inside the file.

Scanned with ACE Scanner

- Use node's inbuilt module fs to read the content of file and print them to the console.

  - readfile :- This will read the File asynchronously.

Code

```
Const fs = require("fs")
fs.readFile("./text.txt", {encoding:"utf-8"}
    ,(err, data) => {
    if (err) {
        console.log("cannot read the file")
        console.log(err)
    } else {
        console.log(data)
    }
})
console.log("Bye Guys!")
```

  - readFileSync :- This will read the file synchronously, until the file reading is not finished, compiler is not going to the Bye Guys!! statement.

Code

```
const fs = require("fs")
const data = fs.readFileSync("./text.txt",
                {encoding: "utf-8"})
console.log(data)
console.log("Bye Guys!!")
```

Scanned with ACE Scanner

- Write into a file using the fs module again.
- While writing it will automatically create the file.

- **writeFile** :- This will write the File asynchronously.

Code

```
Const fs = require("fs")
fs.writeFile("./log.txt", "This is me first time writing in the file", (err) => {
    if (err){
        console.log("Cannot write in the file")
        Console.log(err)
    } else {
        console.log("Data has been written in the file")
    }
})
```

- **writeFileSync** :- This will write the file Synchronously, first the writing will be finished then the next task.

Code

```
Const fs = require("fs")
fs.writeFileSync("./log.txt", "This is me second time writing in the File")
```

Scanned with ACE Scanner

- WriteFile will overwrite the previous content of the File.
- So try appendFile to overcome this.

Code

```
const fs = require("fs")
fs.appendFile("./log.txt", "\n This is me third time writing in the File\n", (err)=>{
    If (err){
        Consale.log("cannot be appended")
        Consale.log(err)
    } else{
        Consale.log("Data has been appended in the File")
    }
})
```

- There is appendFileSync as well, it works in the same as readFileSync and writeFileSync.

Important Note:- Explore https://nodejs.dev/en/learn/ for easier understanding about the node documentation.

Scanned with ACE Scanner

HTTP Basics And Express

What is Server?

- If we just look at the word itself, Server is the one that serves something.
- The system, that is making the request for something, is called the client.
- Server accepts the request from the client and sends the response based on the request.

HTTP

- It stands for Hyper Text Transfer Protocol.
- Set of Protocol or rules that are required do communicate between client and server.
- HTTPS :- It is exactly the same with an added layer of security, it is achieved by SSL (Secure Socket Layer).

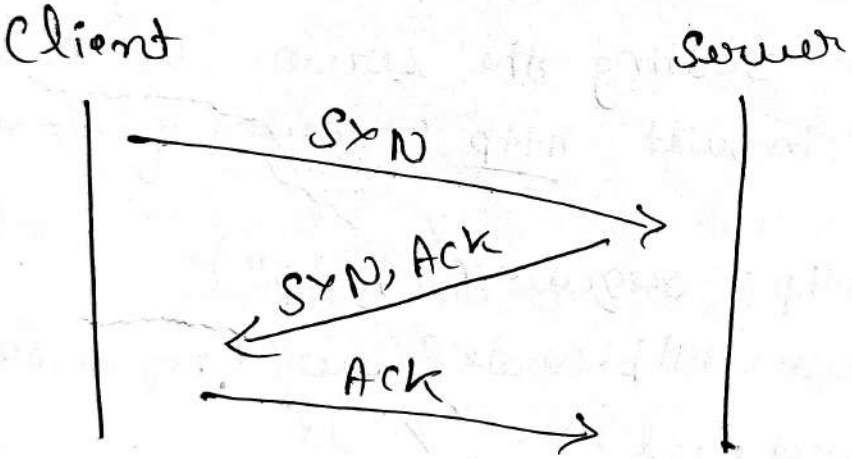
Scanned with ACE Scanner

3 Way Handshake

A 3 way handshake takes place between the client and server, before the actual cycle starts.

- First the client express the intent to the server.
- Server Acknowledges the client's intent.
- Client will tell what is actually needs.

TCP- 3 Way Handshake Process



Bandwidth

- Bandwith is the data transfer capacity of a computer network.

HTTP Verbs / Methods

- GET: To read something from the server.

Scanned with ACE Scanner

- PUT => Modify: Will replace the whole thing.
- PATCH => Modify: Will modify one specific thing in the whall things.
- Delete: To delete something on the server.
- POST: To post/add/sent something to the server.

Creating First Server

- Create a node project by npm init -y.
- Create a file named index.js.
- Now for creating the server we can use the inbuilt http module of node.

Code.

```
const http = require("http")
const server = http.createServer((request, response) => {
  if (request.url === "/") {
    response.end("Hello")
  } else if (request.url === "/reports") {
    response.end("Here are the reports")
  } else if (request.url === "/data") {
    response.end("Data...")
  }
})

server.listen(4500, () => {
  console.log("Listening on the port 4500")
})
```

Scanned with ACE Scanner

- We have programmed our server to give the response as per the request made.
- Whenever we made any changes in server we have to run the server.

.end VS .write

Here we are using .write, Now the client will not know that the response has been ended and it will keep on loading the page, that is why we have to use .end, so that the client knows that response has been ended.

Invalid End Point

- What if the user is making a request to an invalid end point, then we have to take care of that as well while programming our server.

Code

Same as making request but in the end we have to use else

```
else {
    res.end ("Invalid End Point")
}
```

Scanned with ACE Scanner

Send Data from a file

- We can also send other things as a response as well.
- Let us try sending data which is inside a file as a response.

Code

```
const http = require("http")
const fs = require("fs")
const server = http.createServer((req, res)=>{
if (req.url === 'data') {
  fs.readFile("./text.txt", {encoding: "utf-8"},
                                  (err, data) {
  if (err) {
    res.write("No data\n")
    res.end(err)
  } else {
    res.end(data)
  }
  })
}
})
server.listen(4500, () => {
    console.log("Listening on the port 4500")
})
```

Scanned with ACE Scanner

- What are headers? ⇒ It just gives more information about the request or response.
- We also want to send a header as a response.
- This is just to specify what kind of response we are getting.
- Let's pass a header as well.

Code.

```
const http = require("http")
Const fs = require("fs")
const server = http.createServer((req, res) => {
  if (req.url === "/") {
res.setHeader("Content-type", "text/html")
 res.end("<h1> Hello Guys !! </h1>")
  }
})
Server.listen(1500, () => {
    Console.log("Listening on the port 1500")
})
```

Post Request with HTTP

Note :- Install (npm install -g nodemon) and run nodemon index.js to avoid multiple restarting the server.

Scanned with ACE Scanner

- For posting something to the server there should be some data that client has to past while making the request.
- If we go the browser and hit any endpoint, it will show as invalid endpoint as the browser by default make get request.
- Use thunder client for POST.
- We can use POSTMAN that DATA has been recorded.
-

Express

Express is just a framework, that can helps us in creating the server in very easy way.

- In http we were handling various methods and routes, the code was really messy. We can use express to get it rid all of that.
- It is not an inbuilt module of node, so we have to install it using npm.
- Initialise a node project and install nodemon.

Scanned with ACE Scanner

Code.

```
const express = require("express")
const app = express()
const fs = require('fs');
// this is a middle ware
app.use(express.json())

app.get("/", (req, res) => {
    res.send("Hello")
})

app.past("/", (req, res) => {
    consale.log(req.body)
    res.send("data has been accepted")
})

app.past('/addPast', (req, res) => {
    fs.opperdFileSync('./text.txt', `\n ${JSON.
        stringify(req.body)}`)
    res.send('Past requist send sucessfully')
})
```

file :-
text.txt

```
{"name": "BatMan", "Duty": "Fighting Against Crime"}
```

Note:- To implimant this, we have to go to POSTMAN and in body we have to create Past object and then select Post and hit the API.

Scanned with ACE Scanner

Express Middleware & Ecosystem

Advantages of Express

- Helps us to write clean code.
- The biggest advantage is that express has a lot of middleware, even we can create custom one as well to literally do anything.

Middlewares

The middleware in node.js is a function that will have all the access for requesting an object, responding to an object, and moving to the next middleware function in the application request-response cycle.

- It is just a function, which runs once the request is made but before the response is sent.
- What ever the middleware we write, app.use() will use that middle ware for every route below it.

Scanned with ACE Scanner

Code

```
const express = require("express")
const app = express()

// Our first middleware
app.use((req, res, next) => {
    Consall.log("Hello from Middleware")
    next()
})

app.get("contacts", (req, res) => {
    consall.log("Hello from the contacts")
    res.send("Contacts")
})

app.get("/blogs", (req, res) => {
    Consale.log("Hello from the blogs route")
    res.send("Blogs")
})

app.listen(3500, () => {
    Consall.log("Running on 3500")
})
```

Scanned with ACE Scanner

- This function also has the access to request and response object.
- It has next() function as well. => Tells us where to go next.
- If we go and visit the base route, first the middleware is going to be executed then next() will take it to the next thing.
- What will happen if i don't use next()? => try it out.
- The middleware that has been written for all the routes, then keep it at top.
- Try the below code.

Code

```
const express = require("express")
const app = express()

app.use((req, res, next) => {
    Consale.log("Hello from Middleware")
    next()
    Consale.log("Bye from Middleware")
})

app.get("/", (req, res) => {
    consale.log("Hello from the base route")
    res.send("Welcome")
})
```

3)

Scanned with ACE Scanner

```
app.listen(3500, () => {
    consale.log("Running on 3500")
})
```

- This will shaw you haw exactly next() is working.
- Now, lets see the use of request and response object.
- What will happen now?

Code

```
const express = require("express")
const app = express()
app.use((req, res, next) => {
if(true) {
    res.send("BYE")
}else {
  next()
}
})
app.get("/", (req, res) => {
    res.send("Welcome")
})
app.get("/contacts", (req, res) => {
    res.send("Contacts")
app.listen(3500, () => {
    consale.log("Running on 3500")
})
```

Scanned with ACE Scanner

TimeLogger Middleware

- for this create a file with some dumy data.

code :

```
const express = require("express")
const app = express()

app.use((req, res, next) => {
    const startTime = new Date().getTime()
    next()
    const endTime = new Date().getTime()
    console.log(endTime - startTime)
})

app.get("/", (req, res) => {
    res.send("Welcome")
})
app.get("/contacts", (req, res) => {
    res.send("Contacts")
})
app.listen(3500, () => {
    Console.log("Running on 3500")
})
```

- Middleware is just a function, and we can declare it outside as well, and then use it inside the app.use()

Scanned with ACE Scanner

Watch Mon

~~Multiple~~ Middlewares

Code

```
const watchMon = ( req, res, next ) => {
if ( req.url === "/about" ) {
    next()
} else {
  res.send( " Bye " )
}
}
```

Multiple Middlewares

```
const express = require( "express" )
  const app = express()
app.use( ( req, res, next ) => {
    consle.log ( " 1 " )
    next()
    consale.log ( "2" )
})
app.use ( ( req, res, next ) => {
    consale.log( " 3 " )
    next()
    consale.log ( " 4 " )
})
app.get ( " / ", (req, res) => {
      consale.log ( " Home " )
      res.send ( " Welcome " )
})
app.listen (3500, () => {
    consale.log ( " Running on 3500 " )
})
```

Scanned with ACE Scanner

Logger Middleware

Code

```
const logger = (req, res, next) => {
  fs.appendFileSync('./logs.txt', `\n${req.method} + ${req.url}`, "utf-8")
  next()
}
```

- The above middleware will log all the routes visited along with the methods in a separate file just to keep a record.
- We can do anything with the help of middlewares.
- Can I add something to the body of request using middleware while making a post request?

Code.

```
const addStamp = (req, res, next) => {
  req.body.stamp = "Masai Student"
  next()
}
app.post("/addstamp", (req, res) => {
  console.log(req.body)
  res.send("Got the data")
})
```

Scanned with ACE Scanner

Modifying the Request (req) object in Middleware

(1) Adding Custom Properties to req

Code

```
app.use((req, res, next) => {
    req.user = { name: "Himanshu", role:
                 "Admin" }
    next();
});

app.get("/", (req, res) => {
    res.send(`Hello, ${req.user.name}!
You are an ${req.user.role}.`); });
```

(2) Modifying Incoming Request Data

code

```
app.use(express.json()); // Middleware to
                         Parse JSON body

app.use((req, res, next) => {
    if (req.body && req.body.message) {
        req.body.message = req.body.message.
                           toUpperCase();
    }
    next();
});
```

Scanned with ACE Scanner

```
app.post("/message", (req, res) => {
    res.send({ modifiedMessage: req.body.
        message });
});
```

③ Validating Requests and Adding Status

Code

```
app.use((req, res, next) => {
    if (req.headers["x-api-key"] === "secret123") {
        req.isAuthenticated = true; // Custom flag
                                    in req
    } else {
        req.isAuthenticated = false;
    }
    next();
});
app.get("/", (req, res) => {
    if (!req.isAuthenticated) {
    return res.status(403).send("forbidden:
        Invalid API key"); }
    res.send("welcome, authenticated user!");
});
```

Note: There is express inbuild middleware express.json() it just basically parse the json.

Scanned with ACE Scanner

Middleware can also be categorized into three types based on their source and usage :-

(1) Custom Middleware :- This is middleware that you create yourself to handle specific application requirements.

- Example :- A logging middleware in Express.js that records request details.

(2) Inbuilt Middleware :- These are middleware functions provided by frameworks or libraries for common tasks.

- Example : Express.js's express.json() for parsing JSON data in requests.

(3) External Middleware :- These are third-party middleware packages that you install and use for extended functionality.

- Example :- cors for handling Cross-Origin Resource Sharing, morgan for logging HTTP requests in Express.js.

Scanned with ACE Scanner

~~MongoDB Introduction~~

MongoDB Introduction

Databases

- What is Database ? => Database is a place where we can actually store our data.
- Why do we store the data => So that we can use it later, for any kind of purpose
- Best example is we can do a data analysis or we can just store the data of registered users and use it when the registered user wants to login.
- Till now we have stored the data in a file that we have created, that is db.json.
- Why we need a separate database as we can do all those things in a file as well?
- It's because to read, write, delete and update in a file we need to do a lot, which is not an optimal thing to do, before the databases everything we used to be done with files only, but not any more.
- Writing CRUD logic with databases is very easy.

Scanned with ACE Scanner

. All the CRUD logic is (Software) only, which can be carried out by writing a single query.

Types of Data bases

. SQL => Structured query Language.
. NoSQL => Not a Structured query Language.

SQL

. It stands for Structured query Language.
. By the name itself we can see it is structured.

NoSQL

. It stands for Not a Structured query language.
. By the name you can understand that it is not structured.

SQL Vs. NoSQL

. SQL is less flexible.
eg:- My SQL, Oracle SQL, Postgre SQL, MS SQL.
. NoSQL will store data in form of objects.
. NoSQL is flexible
eg:- MongoDB, Cassandra.

Scanned with ACE Scanner

Why Mongo DB ?

- It is very flexible.
- It is very quick and easy to learn Mongo DB.
- It has a syntax similar to Java Script.
- Most Companies these days are using Mongo DB as it is very flexible.

Mongo DB

- It store data in the form of objects.
- Document => Each object in which the data is stored is called document.
- Collection => Group of similar Documents.
- Database => Group of Collections.

DATA BASE

=> COLLECTION

=> DOCUMENT

COMMANDS

-> mongosh -> To start

-> use database -> To create database

-> cls -> To clear the screen

-> Show dbs -> To view the database

-> db. Create Collection ("collection-name") -> To create Collection

Scanned with ACE Scanner

→ show collections → To view all collections of database

→ db.collectionName.insertOne({}) → To create an object or document.

→ db.collection.find() → To find All object associated with that collection.

Note:- Every object has their unique id for a unique identification.

→ db.collectionName.insertMany([{},{}]) → To create Multiple object

→ db.collectionName.insertMany([{},{}]) → To create Many objects.

CREATE → READ → UPDATE → DELETE

→ To filter :- You can use any key-value of that object, that you want to filter.

→ Let us say you have many object of different-different key-value pair and you want to filter the object (document) related to their name; so you can :-

→ db.fictional.find({name: "Any name"})

(collection name.

Scanned with ACE Scanner

→ Update (Put / Patch) :-

Any filter attribute

① db. fictiond . updateOne ({ price : 100 }, { $set : { data : "10-03-2024" } })

→ It put data in first object of price : 100

→ first object means it can add date in first object, no matter other object have also some key-value pairs (price : 100).

you can use other key-value pairs as well

② db. fictiond . updateMany ({ price : 100 } , { $set : { date : "10-03-2024" } })

→ It put date in all object whose key-value pair similar like { price : 100 }.

Output . When you hit Enter

```
{
    acknowledged : true,
    inserted Id : null,
    matched Count : 4 ( No. of documents )
    modified Count : 4 ( No. of documents )
    upserted Count : 0
}
```

→ Why inserted Id : null ?

→ Because Mongo DB ne phle hi object ke id de rakhi the, to update karne ke time baar-baar id nahi de sakte the esliya null likha.

Scanned with ACE Scanner

Note :-

① to create (insert) - one and many { } , [ ]

② to read - find = { filter } , ( )

③ to update - one or many = { filter , what to update }

④ to delete - one or many = { filter }.

Comparison & Logical Operators

<=	→	lte	!= → ne
>=	→	gte	&& → and
<	→	lt	|| → or
>	→	gt	
==	→	eq	

How to use comparison & logical operators in our db.

→ db.collectionName.find ({ health : { $eq : 4033 })

You can replace the operators with other operater as well.

How to use && and ||.

→ db.collectionName.find ({ $and : [{ health : [{ health : { $lt : 6033 } , { health : $gt : 4033 }] }] })

You can replace with or also.

Scanned with ACE Scanner

Sort, skip and limit

① .sort ({ filter : 3 }) => 1 ascending and -1 descending.

eg:- db.superhero.find().sort({ health : 1 3 })

→ It will sort the document in ascending order.

→ If ({ health : -1 3 }), it will sort the document in decending order.

② skip and limit

→ db.superhero.find().skip(2).limit(2)

→ It will skip two document and show next 2 document.

→ db.superhero.find().limit(10)

→ It will show the top 10 documents.

Use :- It use in pagination

50 Documents

One Page : 10 documents

Page 1: skip(0).limit(10)

Page 2: skip(10).limit(10)

Page 3: skip(20).limit(10)

Page 4: skip(30).limit(10)

Scanned with ACE Scanner

Aggregation in Mongo DB

Key TOPICS Discussed in Today's class

- Aggregation & Aggregation Pipeline
- $ match
- $ project

Mongo DB Aggregation frome work

The Mongo DB Aggregation fromework is a Pawerful tool for perfarming complex data transfarmations and analyses directly within Mongo DB database. It is designed to aggregate data from multiple documents and return computed results, allawing for operations like data summarization, transfarmation, and complex filtering.

Understanding the Aggregation Pipeline

1. Basics of the Aggregation Pipeline :-

. The aggregation Pipeline is a fromework that processes data records and returns computed results.

Scanned with ACE Scanner

. It operates through a series of stage, each performing an operation on the data (e.g., filtering, grouping, projecting).

. Data flows sequentially from one stage to the next, with the output of one stage serving as the input for the subsequent stage.

In simple :-

pipeline :- the output of query 1 becomes input of query 2

combine the queries

eg :- ~~get~~

```
{ name : "Himanshu",
  age : 15
}
```

-> get age field and i want to multiply it with 5.

Scanned with ACE Scanner

→ $Match is similar to find() but the difference is we can't update with find() but we can do update and some performance with the use of $match.

Commands

$match

① Get the age gte 30.

→ db.customers.aggregate([{ $match: {age: {
$gte: 30 }}}])

② Get all the users whose age is in b/w 18-60.

→ db.customer.aggregate([{$match: {$and: [{$
and: [{age: {$gte: 18}}, {age: {$lte: 60}}]}]})

$ Project

Shape or transform document according to our specification,

① Select any field
② We can rename a field {myname: "Mohsin"}
③ Compute certain field.

{ name: "Mohsin",
age: 15
}

Scanned with ACE Scanner

→ Select any field

→ db.customers.aggregate([{$project:{email:1,age:0}}])

→ This will not work because we are confusing mongodB to show email and not show age (If "$project:{email:1}" email show and all other field not show then what is the need of writting age:0, It make no sence.

→ db.customers.aggregate([{$project:{email:1,age:1}}])

→ This means email and age field will be shown eslse other field will not shown.

→ db.customers.aggregate([{$project:{email:0,age:0}}])

→ This means email and age field will not shown else other ~~field~~ field will show.

Note:- What db.customers.aggregate([]) signifies?

→ It Initiates pipeline aggreation on customers collections.

Scanned with ACE Scanner

→ Transform age from years to months and include first_name :

d. customers.aggregate([
{$project:{NewAge:{$multiply:["$age",12]},
first_name:1}}]).

Combining fields :-

· Concatenate first_name and last_name into a full name :

→ db.customers.aggregate([
{$project:{Name:{$concat:["$first_name"," ","$last_name"]},age:1}}])

filter by Age and Project Names

$match and $Project ⇒ pipeline

Print first_name and last_name of the customers whose age is greater than 20.

→ age is greater than 20 ⇒ $match ⇒ Output

→ Print first_name and last_name of the customers ⇒ Input

Scanned with ACE Scanner

```
db.customers.aggregate([
    { $match: { age: { $gt: 20333,
    { $project: { first_name: 1, last_name: 1 } }
])
```

Advanced Usage: Updating Nested fields

- Scenario:- Updating the zipcode in a user's address. This operation demonstrates how to target and update nested fields within documents, showcasing the flexibility of MongoDB in handling complex data structures.

```
db.users.updateOne(
    { "username": "john_doe" },
    { $set: { "address.zipcode": 12345 " } })
```

Student Activity

- Task: Convert salary to thousands and append "k" (e.g., 35k for a salary of 35167).
- This activity encourage the application of multiple operations like division rounding, and string manipulation with an aggregation pipeline.

Scanned with ACE Scanner

Aggregation — II

Third stage $group:

→ Combines or groups the documents based on specific criteria.

→ Allows performing Aggregation operation like sum, count, average etc.

→ Useful for generating statistics or analyzing data.

Grouping data based on specific criteria (colour field)

red : 3
yellow : 4
blue : 5

unique in mongodb:

1. _id : colour(red) => {

```
"name" : "John Smith",
"age" : 35,
"Salary" : 50000
},
,,
,,
}
```

Scanned with ACE Scanner

① Give all the unique genders from the db.
specific criteria : gender.

→ db.customers.aggregate([{$group:{_id:"$gender"}}])

Output :-

```
[{_id: 'Non-binary'},
 {_id: 'Bigender'},
 {_id: 'Agender'},
 {_id: 'Genderqueer'},
 {_id: 'Female'},
 {_id: 'Genderfluid'}]
```

Why this "_id"?

1. Grouping criteria "gender"
2. unique indentifier : "gender" → "Bigender" || Agender || ~~Ao~~ Female.

Aggregation operation like sum, avg etc.

2. Give me all the unique gender and the number from each gender group?

use ⇒ group, count → it will give me count of documents

Scanned with ACE Scanner

→ db.customers.aggregate([{ $group: { _id: $gender }}])

→ db.customers.aggregate([{
$group: { _id: "$gender", UserCount: { $count: {} } }
}])

③ Give me all the unique genders and their avg age of each gender.

use ⇒ group, $avg ⇒ average of input

→ db.customers.aggregate([{
$group: { _id: "$gender", AverageAge: { $avg: "$age" } }
}])

④ Give me avg age of the customers.

⇒ observation → unique indentifier = NO

⇒ specific criteria as unique indentifier "$gender"
→ represents a field.

" " → simple string or a simple number like 1, 2, 3, 4, 5.

⇒ We can perform aggregate operations

Scanned with ACE Scanner

which specifically mentioning group criteria.

→ We will make a single group by using hardcoded string as a number instead of dynamic string ? like ("$ge

db.customers.aggregate([{$group:
{_id:1, AverageAge:{$avg:"$age"}}}])

↳ you can use (" ") → String
↳ write any string.

⑥ Group by first name and age :-

→ id as unique identifier => first name and age.

→ No. of group will be equal to number of documents.

→ d.b.customers.aggregate([{$group:{_id:{name:"$first_name", age:"$gender"}}}])

Note :- don't use array because i have to relay on indexes.

→ db.customers.aggregate([{$group:{id:["$first_name","$gender"]}}])

Scanned with ACE Scanner

forming Pipelines:-

⑦ I wont to get the unique gender of these customers who are above the age or equal to 50 and their avg salary.

⇒ filter out the customers ⇒ match age gte 50 years

⇒ Group them by gender

⇒ Calculate their avg salary.

→ db.customers.aggregate([{
$match:{age:{$gte:50}}}, {$group:{_id:"gender", AverageSalary:{$avg:"$salary"}}}])

---

⑧ I wont to get all the unique genders and the average salary, where avg sal is >= 25k

⇒ group customers based on gender.

⇒ Calculate their avg salaries per group.

⇒ match the calculated avg >= 25k

→ db.~~customers.aggregate([{$match:{age:{~~

→ ([{$group:{_id:"$gender", AverageSalary:{$avg:"$salary"}}}, {$match:{AverageSalary:{$gte:25000}}}

Scanned with ACE Scanner

(9) Print the full name of all those customers whose age is >= 40

Soln 1 :-

→ filter customers age >= 40 ($match)

→ concat first_name and last_name ($project)

Soln 2 :-

→ concate first_name and last_name

→ filter customers age >= 40.

Scanned with ACE Scanner

Mongo DB Relationships

→ What is a Database?

→ Place where we store data in a organized manner.

→ Why do we actually need a database?

→

1. Security :- Agar tumhare pass koi sensitive data hai, jaise ki users ke passwords ya transactics, toh tumhe security chahiya. Database tumhe authorized access dena ka option deta hai.

- Read Access - Kuch users sirf data dekh sakein, par modify na kar sakein.
- Specific Rights - Kise particular user ko sirf read, write, update ya delete ka access dena ho.
- Retrieve Deleted Data - Agar galti se koi data delete ho gaya, toh database se wapas la sakte ho.

Scanned with ACE Scanner

② Flexibility & Optimization

Agar tumhare pass 50-60 GB ka data hai aur tum usko file system me store kar rahe ho, toh queries chalane me bahut time lag sakta hai.

File System - 1 minute lag sakta hai ek query complete hone me.

Optimized Database - Sirf 10ms me kaam ho jayega.

③ SQL vs NoSQL -

SQL Database - Structured format me data store karta hai (tables ke form me).

NoSQL Databases - Unstructured ya semi-structured data ke liye better hote hai (jaise JSON ya documents).

④ Storage Capacity.

Agar tumhare pass bahut zyada data store karna hai, toh file system se manage karna tough ho jata hai. 1000 GB, 500 GB, 200 TB etc.

Database efficiently store aur retrieve karne me help karta hai.

Scanned with ACE Scanner

5. Scalability

Man lo tumhari website me 500 users hain, toh ek database handle kar lega. Lekin agar users 5000 yo 50k ho gaye, toh queries bhi badh jayengi.

500 users → 1 database → 200 queries/sec
5000 users → Scalable server → 2000 queries/sec (+lotime)

Agar database scalable nahi hoga, toh slow system slow ya crash ho sakta hai.

6. Query Operation (CRUD)

File system me CRUD operations karna complex hota hai!

1. Pehle file read karni padti hai.
2. fir usme loops ya HOFC (Higher Order functions) use karke data dhundhna padta h.
3. Time complexity bhi zyada hoti hai!

Database queries ka use karke yeh sab fast ho jata hai!.

7. Persistent Storage.

Database ek permanent storage hai. Matlab agar system band bhi ho jaye, toh bhi data safe rahega.

Scanned with ACE Scanner

file - based systems → Corruption ya data loss ka risk.

- Databases → Secure, scalable, fast aur optimized.

Q What is MongoDB?

MongoDB ek NoSQL Database hai, jo documents (JSON-like data) me store hota hai.

- flexible Schema → Tables ki tarah rigid structure nahi hota.
- High Performance → Read/write fast hota h.
- Scalability → Large scale apps ke liye best.

CRUD Operations in MongoDB.

CRUD ka full form:

- C - Create → insertOne(), insertMany()
- R - Read → find(), findOne()
- U - Update → updateOne(), updateMany()
- D - Delete - deleteOne(), deleteMany()

MongoDB Aggregations (Data Processing & filtering)

Scanned with ACE Scanner

MongoDB aggregation queries ka use filtering, grouping, reshaping, aur complex data analysis ke liye hota hai.

Main Aggregation Stages :-

- $match → Data filter karna.
- $group → Similar data ko group karna
- $project → Documents ko reshape karna.

What, Why & How of Relationships in Db?

Relationships → Jab ek data ka doosre data se connection hota ha, toh usko relationship kehte hai.

1. One-to-One Relationship :- Ek user ko ek hi blog ho sakta hai.

Code

{ user : "Alice", id : 1 }

{ postTitle : "Learn React", postId : 2, authorId : 1 }

2. One to Many / Many-to-One Relationship

Ek user ke multiple post ho sakta hai.

Users Collection :- { user : ", id : 1,

associated Posts : [{ PostsId : 2 }, { PostsId : 3 },

{ postsId : 4 }, { postsId : 5 }] }

Scanned with ACE Scanner

Many-to-Many Relationships

Ek student multiple courses me enrolled ho sakta hai & ek course me multiple students ho sakte hai.

Course Collection

Code

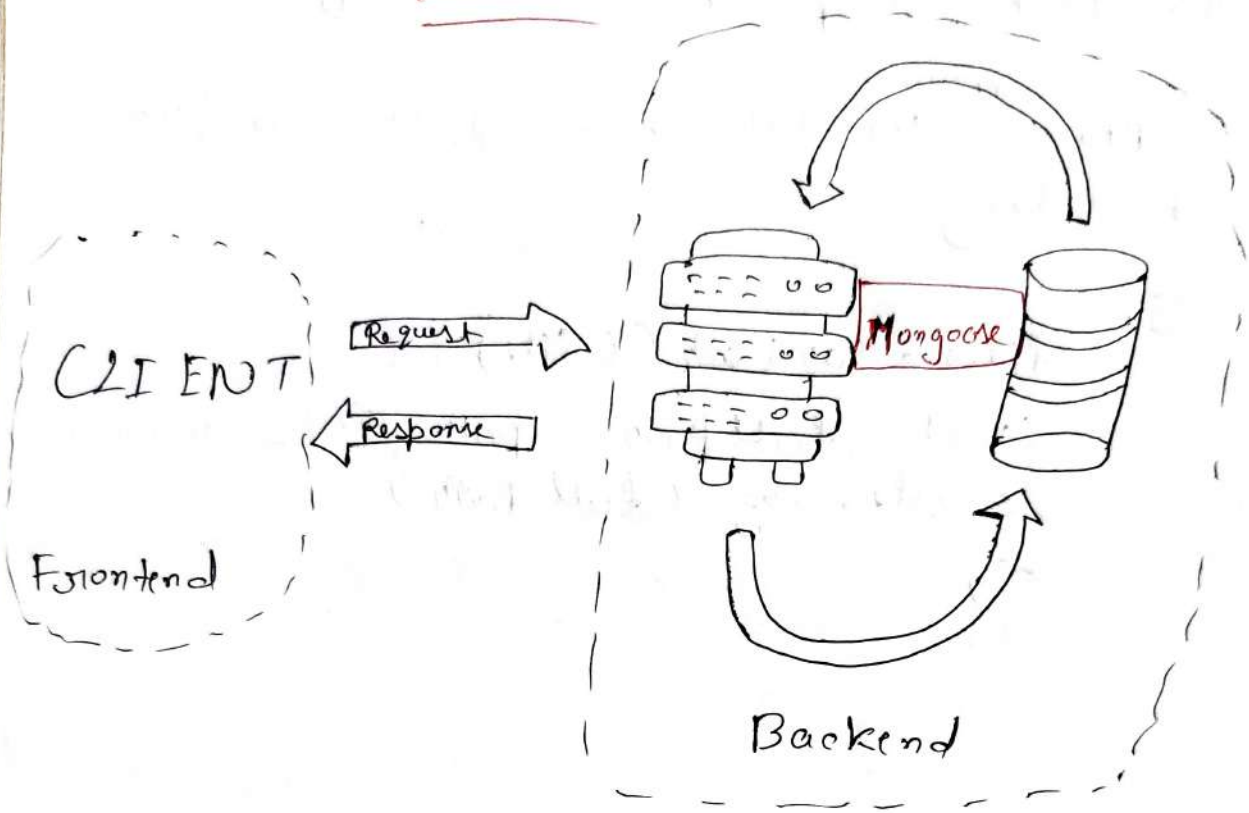
```
[
  { course: "Hindi", id: 1, studentsEnrolled:
      ["Alice", "Bob"] },
  { course: "English", id: 2, studentsEnrolled:
      ["Alice"] }
]
```

Students Collection

```
[ { name: "Alice", id: 5, enrolledCourses: [
      "Hindi", "English"] },
  { name: "Bob", id: 6, enrolledCourses:
      ["Hindi"] }
]
```

Scanned with ACE Scanner

Mongoose



Flow of a Request and Response

Q① What is Mongoose?

Ans Mongoose is a ODM (Object Data Modeling) library for MongoDB and Node.js.

Why use it?

- Easy schema-based modeling
- Validation & type checking
- CRUD helpers
- Middleware support

Scanned with ACE Scanner

Setup :- npm install mongoose

Connect to Mongo DB

Code

```
const mongoose = require("mongoose");

const dbConnection = async () => {
try {
await mongoose.connect("mongodb://127.0.0.1")
 consale.log("Connected to DB successfully");
} catch (error) {
 consale.log("DB connection failed:", error);
 }
};
```

Note :- Call this function inside your app.listen() block.

Schema & Model

Define a Schema

Next Page →

Scanned with ACE Scanner

Code

```
const UserSchema = mongoose.Schema({
name : { type : String , required : true },
email : { type : String , required : true, unique : true },
userName : { type : String , required : true, unique : true },
age : { type : Number
password : { type : String , required : true },
pan : { type : String },
profilePicture : { type : String },
city : { type : String , enum : ["Pune", "Bengaluru",
                                 "Delhi"] }
});
```

Create a Model

```
const UserModel = mongoose.model("userProfile",
    UserSchema);
```

POST API - Add User

Code

```
app.post("/addUser", async (req, res) => {
    const newUser = req.body;

    try {
    await UserModel.create(newUser);
    res.status(200).send("User added succefully");
} catch (error) {
    res.status(500).send({message : error});
}
});
```

Scanned with ACE Scanner

Server Error Handling

MongoServer Error: E1000 duplicate key error
This happens when email or userName field (which are unique) already exist in DB. entries.
• Fix: Ensure unique values while creating new

Get - fetch All Users

Code

```
app.get("/users", async (req, res) => {
  try {
    const users = await UserModel.find();
    res.status(200).send(users);
  } catch (error) {
    res.status(500).send({message: error.message});
  }
});
```

Put - Update User by ID

Code

```
app.put("/updateUser/:id", async (req, res) => {
  const {id} = req.params;
  const updateData = req.body;

try {
const updatedUser = await UserModel.findByIdAndUpdate(id, updateData, {
    new: true, // Return updated user
});
```

Scanned with ACE Scanner

```
if (! updatedUser){
return res.status(404).send({message:
        "User not found"});

}
res.status(200).send({message:"User updated",
        user: updatedUser});

} catch (error) {
res.status(500).send({message: error.message});
}
});
```

Delete – Remove User by ID

code

```
app.delete("/deleteUser/:id", async (req, res) =>{
  const {id} = req.params;

try {
const deletedUser = await UserModel.findByIdAnd
                    Delete(id);

if (! deletedUser){                          found"});
  return res.status(404).send({message:"User not
}
res.status(200).send({message: "User deleted
                    Successfully"});
} catch (error) {
  res.status(500).send({message: error.message});
}
});
```

Scanned with ACE Scanner

Authentication (Bcrypt And JWT)

Required Packages :-

```
const bcrypt = require("bcrypt"); // Password hash
const jwt = require("jsonwebtoken"); // Token generation
```

What is Authentication ?
Authentication ka matlab hota hai :
"User ka identity verify karna" — jese ki email/password check karna login ke time .

BCRYPT — Password Security

. Why use Bcrypt ?
✓ Secure way to store passwords
✗ Don't store plain passwords in database

→ Hashing Password (Registration time)

Code
```
bcrypt.hash(password, SaltRounds, callback)
```

. password : user input
. SaltRounds : Complexity (2 se 10 thik hota hai)
. Callback : hashed password milta hai yaha.

Scanned with ACE Scanner

code

```
bcrypt.hash(password, 2, async(err, hashedPassword) => {
    // Save hashed Password to DB
});
```

→ Compare Password (login time)

code.

```
bcrypt.compare(plainPassword, hashedPassword from DB, callback)
```

Code Example

```
bcrypt.compare(password, user.password, (err, result) {
    if (result) {
        // Password match!
    }
});
```

JWT – JSON Web Token

⇒ Why JWT?

→ Jab user login kare, usko ek token milta hai
→ Ye token proof hota hai ki "ye banda authorised hai".

• Generate Token (on login)

→ jwt.Sign(payload, secretkey)

Scanned with ACE Scanner

Code Example.

```
const token = jwt.sign({ roleAssigned: user.role }, "masai");
```

→ Verify Token (middleware)

code:

```
jwt.verify(token, secretkey, callback)
```

code.

```
const token = req.headers.authorisation;
jwt.verify(token, "masai", function(err, decoded){
  if(err){
    return res.send('Access Denied');
  } else {
    next();
  }
});
```

Flow Summary:

1. Register:
   - Password ko bcrypt se hash karo → DB me store karo.
2. Login:
   - Password compare karo → agar sahi hai → JWT token generate karo
3. Middleware:
   - Har secured route pe token verify karo → agar sahi → next()

Scanned with ACE Scanner

Secure Route Example :-

```
app.get('/report', authMiddleware, (req, res) => {
    res.send("Report sent successfully")
});
```

Tips :-

- bcrypt.hash(password, saltRounds) = password encryption
- jwt.sign(payload, secret) = token banata hai
- jwt.verify(token, secret) = token valid hai ya nahi check karta hai.

Scanned with ACE Scanner

Authorization

Authorization kya hota hai?

Jab user login kar leta hai (authenticated), uska baad check hota hai:

- Ye user admin hai ya normal user?
- Kya is user ko ya route access karne ki permission hai?

Kaise implement karte hain (Role-based Authorization)

1. User ke object me ek role hone chahiye (admin / user)

Already tumhare model me ye hai:

Code:

```
role : { type: String, required: true } // "admin" or "user"
```

2. Jab JWT token generate karte ho login pe, role usme daal do:

code:

```
const token = jwt.sign({ role: user.role, userId: user._id }, "masai", { expiresIn: "1h" });
```

Scanned with ACE Scanner

3. Create authorization middleware (based on role):

Code.

```
const authorize = (allowedRoles) => {
  return (req, res, next) => {
    const token = req.headers.authorization;
    if(!token) return res.status(401).send("Token
                                    missing");

    jwt.verify(token, "masai", (err, decoded) => {
    if (err) return res.status(403).send("Invalid
                                    token");

    // Check if user role is allowed
if (allowedRoles.includes(decoded.role)) {
    req.user = decoded;
    next(); // allow access
} else {
  res.status(403).send("You are not authorized
                                    to access this");

  }
  });
};
```

Scanned with ACE Scanner

④ Use it in routes :

Code.

```
// Only admin can access
app.get("/admin-dashboard", authorize(["admin"]),
                                (req, res) => {
  res.send("Welcome to Admin Dashboard");
});

// Only user can access
app.get("/user-dashboard", authorize(["user"]),
                                (req, res) => {
  res.send("Welcome to User Dashboard");
});

// Bath admin and user can access
app.get("/common", authorize(["admin", "user"]),
                                (req, res) => {
  res.send("Accessible to both");
});
```

Note :

- Authentication ensures you are logged in,
- Authorization ensures you are allowed to do this.

Scanned with ACE Scanner

Full Stack II (Backend + Frontend)

Step-by-step: Local Project ko Github pe Push karna (without node_modules)

1. .gitignore file check karo

Tumhara .gitignore mein ye line hona chaiye:

code

```
node_modules/
.env
```

2. Git hub pe repo banao bina Readme dale.
3. Terminal pe Readme file banao.
4. git init
5. git add .
6. git commit -m "Initial commit"
7. git remote add origin https://....git
8. Push karo main branch pe:

```
git branch -M main
git push -u origin main
```

* first time push mein -u lagana zaroori hai, taki local main link ho jaaye remote main se.

Scanned with ACE Scanner

Deployement of backend :

1. Cyclic (evaluation ke liya best)
2. Railway
3. Render

How to use .env variables in your index.js, follow these simple steps :-

1. Install dotenv package
   In your termind, run :-
   npm install dotenv

2. Create a .env file in your root folder
   Inside .env, you can define your variables like this

```
PORT = 8080
MONGO_URL = mongodb://localhost:27017/notesapp
JWT_SECRET = masai
```

3. Import and configure dotenv at the top of your index.js
   Add this line at the very top of index.js.
   Code.

```
require('dotenv').config();
```

Scanned with ACE Scanner

④ Use the variables in your code like this :-

Replace hardcoded values with process.env values.

code eg:

```
const PORT= process.env.PORT || 8080 ;

const connectdb = () => {
    mongoose.connect (process.env.MONGO_URL);
    }

var token = jwt.sign (
    { userId: user._id, userName: user.name },
    process.env.JWT_SECRET
);
```

Note:-

- Do not commit your .env file to Github. Add it to your .gitignore.

How to deploy our backend in Render.

→ Go to www.render.com website
→ Sign in using Github
→ Go to Web Services
→ Go to Public Git Repository option and Past HTTPS URL from Github repository

Scanned with ACE Scanner

→ You will see lot of options

- ① Root Directory → / or leave empty
② Build Command → npm install
③ Start Command → node server.js (index.js)
④ Write Environment variables
  - PORT = 8080
  - MONGOOSE_URL = URL (Atlas URL!)
  - JWT_SECRET = Your Secret key

CORS kya hota ha ?

CORS ka full form hai Cross-Origin Resource Sharing.

Yeh ek browser ka security feature hota hai jo prevent karta hai ki ek website (ya frontend) dusre origin (jais backend) se data na le bina permission ke.

Problem kaha aati hai ?

Agar aap React frontend aur Express backend alag ports pe chala rahe ho :-

- React app: http://localhost:3000
- Express server: http://localhost:8080

To jab React se request Express server pe jaayegi, browser us request ko block kar dega

Scanned with ACE Scanner

- kyunki wah "cross-origin" hai.

Solution - cors package

Is problem ko solve karne ke liye, hum cors middleware use karte hain Express app mein.

Kaise use karein:

① Install karo :-

code:

```
npm install cors
```

② Import aur use karo Express mein:

code:

```
const cors = require('cors');
app.use(cors());
```

Ab aapka backend cross-origin request accept karega.

Optional: Sirf ek specific origin allow karna ho :-

code:

```
app.use(cors({
  origin: 'http://localhost:3000', // sirf is
          origin se request allow karo
  method: ['GET', 'POST', 'PUT', 'DELETE'],
  credentials: true
}));
```

Summary :- Jab frontend aur backend alag origin pe chalte hain (3000 vs 8080), to browser security ke wajah se request block ho jaata hain. cors() use karne se Express server dusre origin

se koyi request ko accept kar leta hai.

Notes.

→ Use Mongoss Mongoose Atlas connection string in your backend app for deployment as well as common use.

→ You can us Atlas string in your mongoose compass to connect with Atlas database and use in mongodB compass.

→ Make sure your package.json include a start script.

We should include "start": "node index.js" for deployment (like Render, Heroku, etc.), and "dev": "nodemon index.js" only for development on your local machine.

final Recommended Setup in package.json :

code

```
"scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js"
}
```

Scanned with ACE Scanner